

Yes. ****Before starting Stage 1, I recommend setting up a clean RAG learning environment with `uv`**.**

Since this is a learning journey that will eventually move toward production-grade RAG, I suggest we create ****one dedicated RAG workspace**** and manage dependencies through `uv`, rather than installing packages globally.

1. Target environment

```
rag-learning/  
├── .env/                  # uv-managed virtual environment  
├── src/  
│   ├── rag_learning/  
│   │   └── __init__.py  
│   └── data/  
│       ├── raw/          # Original documents  
│       ├── processed/    # Processed documents  
│       └── sample/       # Learning/sample documents  
├── notebooks/           # Experiments  
├── tests/  
├── .env                  # Local environment variables  
├── .gitignore  
├── pyproject.toml  
├── uv.lock  
└── README.md
```

The important thing is that ****we don't need all RAG packages on Day 1****.

We'll install dependencies ****stage by stage**** as we learn.

2. Why `uv`?

For this journey, `uv` is a good choice because it handles:

```
Python version  
  ↓  
Virtual environment  
  ↓  
Package installation  
  ↓  
Dependency resolution  
  ↓  
Lock file  
  ↓  
Reproducible environment
```

We'll primarily work with:

```
uv  
├── uv init  
├── uv python  
├── uv venv  
├── uv add  
├── uv remove  
├── uv sync  
└── uv run
```

This also gives you experience with a modern Python project structure rather than simply doing:

```
pip install ...
```

3. Step 1 — Check Python

Open PowerShell and run:

```
python --version
```

Also check:

```
py --version
```

For this learning journey, I recommend **Python 3.12 or 3.13**.

You previously used Python 3.13 in your GenAI project, so we can continue with that if it's available.

4. Step 2 — Check `uv`

Run:

```
uv --version
```

If you already have it:

```
uv 0.x.x
```

you're ready.

If not, install `uv` using the official installer.

uv official installation documentation: <https://docs.astral.sh/uv/getting-started/installation/>

Then restart PowerShell and verify:

```
uv --version
```

5. Step 3 — Create the RAG learning project

Navigate to the location where you keep your learning projects.

For example:

```
cd C:\Users\<your-user>\Documents
```

Create the project:

```
uv init rag-learning
```

Then:

```
cd rag-learning
```

You should get something similar to:

```
rag-learning/  
■  
■■■ .gitignore  
■■■ .python-version  
■■■ README.md  
■■■ pyproject.toml
```

6. Step 4 — Select Python version

I recommend explicitly selecting Python 3.13:

```
uv python pin 3.13
```

Check:

```
uv python list
```

You should see the available Python installations.

The ``.python-version`` file will record the selected version.

7. Step 5 — Create the virtual environment

Run:

```
uv venv
```

This creates:

```
rag-learning/  
■  
■■■ .venv/
```

You don't necessarily need to manually activate it when using `uv``.

For example, instead of:

```
.\.venv\Scripts\activate  
python main.py
```

you can use:

```
uv run python main.py
```

This is one of the nice things about `uv``.

8. Step 6 — Create our learning directories

Run:

```
mkdir src  
mkdir data  
mkdir notebooks  
mkdir tests
```

Then:

```
mkdir src\rag_learning  
mkdir data\raw  
mkdir data\processed  
mkdir data\sample
```

We'll eventually have:

```

rag-learning/
├── .venv/
├── src/
│   └── rag_learning/
│       ├── data/
│       │   ├── raw/
│       │   ├── processed/
│       │   └── sample/
│       ├── notebooks/
│       ├── tests/
│       ├── .gitignore
│       ├── .python-version
│       ├── pyproject.toml
│       └── README.md

```

9. Step 7 — Add the first dependencies

I **don't** recommend installing LangChain yet.

For Stage 1, we're primarily learning:

> **Data Ingestion**

So initially we'll need general document-processing capabilities.

I'd start with:

```
uv add pypdf python-docx
```

This gives us:

```

pypdf
  ↓
PDF ingestion

python-docx
  ↓
Word document ingestion

```

Later, as we encounter other document types, we'll add their dependencies.

For example:

```

HTML
  ↓
beautifulsoup4

Markdown
  ↓
markdown

Excel
  ↓
openpyxl

CSV
  ↓
pandas

```

We don't need to install everything now.

10. Step 8 — Add development dependencies

For testing:

```
uv add --dev pytest
```

Later we'll add tools such as:

```
ruff
mypy
pre-commit
```

but again, **don't install everything at once**.

We're learning RAG, not package management.

11. Step 9 — Verify the environment

Run:

```
uv sync
```

Then:

```
uv run python --version
```

And:

```
uv run python -c "import pypdf; print('pypdf OK')"
```

Also:

```
uv run python -c "import docx; print('python-docx OK')"
```

If both work, our initial environment is ready.

12. Step 10 — Configure `pyproject.toml`

`uv` will maintain the dependencies for us.

You'll have something conceptually like:

```
[project]
name = "rag-learning"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "pypdf",
    "python-docx",
]

[dependency-groups]
dev = [
    "pytest",
]
```

Don't manually maintain dependency versions unless necessary.

Let `uv` manage them:

```
uv add package-name
```

and:

```
uv remove package-name
```

13. Step 11 — Git setup

Since you're already learning Git and GitHub, I strongly recommend putting this RAG learning project under Git from the beginning.

From the project root:

```
git init
```

Then:

```
git status
```

Your `.gitignore` should contain at least:

```
.venv/  
__pycache__/  
*.pyc  
.env  
.pytest_cache/  
.ipynb_checkpoints/
```

Importantly:

```
.venv/
```

should **never** be committed.

But:

```
uv.lock
```

should be committed.

The lock file helps reproduce the same dependency environment.

14. Our dependency strategy

This is particularly important for your learning journey.

We're **not** going to create one huge environment containing 50+ RAG libraries.

Instead:

```
Stage 1  
↓  
Install Stage 1 dependencies  
  
Stage 2  
↓  
Add Stage 2 dependencies
```

Stage 3
↓
Add Stage 3 dependencies

...

Stage 6
↓
Add Retrieval dependencies

For example:

Stage 1
■■■ pypdf
■■■ python-docx
■■■ beautifulsoup4

Stage 3
■■■ chunking libraries
■■■ tokenizer

Stage 4
■■■ sentence-transformers
■■■ embedding-related packages

Stage 5
■■■ chromadb
■■■ vector database clients

Stage 6
■■■ retrieval libraries
■■■ BM25
■■■ reranking models
■■■ hybrid retrieval components

This approach will make it much easier for you to understand **why each package exists**.

15. One important rule for our RAG journey

I recommend that we follow this rule:

> **First understand the concept → implement it with native Python → then implement it with LangChain.**

For example, when we reach Retrieval:

Understanding
↓
Python implementation
↓
Vector DB native API
↓
LangChain Retriever
↓
Advanced RAG

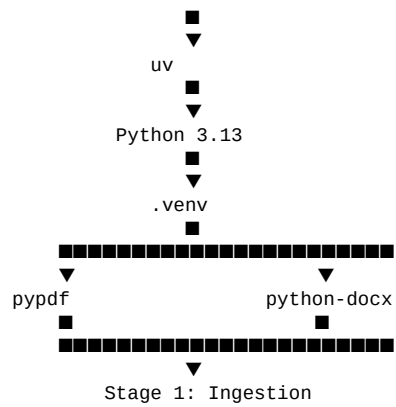
This prevents the common problem of learning RAG as:

```
retriever = vectorstore.as_retriever()
```

without actually understanding what's happening underneath.

Our immediate goal

Before we start **Stage 1 — Data Ingestion**, let's get this foundation working:



****I recommend we do this setup interactively, one step at a time**, rather than creating everything blindly. Start with:**

```
uv --version
python --version
```

and share the output. Then we can establish the exact Python/`uv` setup on your Windows machine before creating the project.